AI Prompt Templates for Engineering Work

Reusable templates for using an AI assistant as an engineering partner. They prioritise specification, system context, domain judgement, verification, and clear ownership over raw code-generation speed.

Inspired by The Developer Career Ladder Is Being Rewritten: the useful human contribution is increasingly to frame the problem well, assess trade-offs, verify output, and own the result.

Use This Structure

Replace bracketed fields before sending a prompt.

```
Goal: [desired user or business outcome]
Context: [system, relevant files, platform, constraints]
Non-goals: [what must not change]
Evidence: [logs, tests, screenshots, links, measurements]
Definition of done: [observable acceptance criteria]
Output: [analysis, patch, tests, decision record, etc.]
```

Always ask for uncertainty to be stated. Never include credentials, private customer data, signing material, or unredacted production logs.

1. Diagnose Before Editing

```
Act as a senior engineer diagnosing a regression.

Goal: explain why [observed behaviour] occurs.
Context: [platform], [relevant component/files], and [last known-
good version].
Evidence: [reproduction steps, logs, screenshots, stack trace].

Trace the complete state and lifecycle path. Compare the last
known-good implementation if practical. Distinguish evidence from
inference.
Do not propose a patch until you state:
1. the root cause,
2. the invariant that is broken,
3. the smallest safe repair,
4. platform-specific risk, and
5. how to verify the repair.
```

2. Write an Implementation Specification

```
Turn this request into a small implementation specification:
[request]

System context: [architecture, affected platforms, existing
abstractions].
Constraints: preserve [behaviours], avoid
[APIs/dependencies/refactors].

Return:
- problem and user impact,
- proposed behaviour and explicit non-goals,
- touched components and data/state ownership,
- acceptance criteria in Given/When/Then form,
- edge cases and rollback path,
- the minimum verification level.

Flag missing information instead of inventing requirements.
```

3. Plan a Minimal, Safe Patch

```
Given this approved specification:
[specification]

Inspect only the files needed to implement it. Propose the
smallest patch that preserves existing architecture and public
behaviour.

Before editing, list the exact files and why each is necessary.
After editing, report:
- what changed and what intentionally did not,
- platform impact,
- accessibility impact for UI changes,
- tests/builds to run,
- remaining risks or assumptions.
```

4. Architecture and Trade-off Review

```
Evaluate these options for [decision]:
[option A]
[option B]
[option C, if applicable]

Context: [scale, users, reliability/security/privacy constraints,
team skills].
Decision criteria: [latency, complexity, operating cost,
reversibility, delivery time].

Use a concise decision table. For each option, name the failure
modes and mitigations. Recommend one option, state the trade-off
being accepted, and write a short ADR-style decision record.
Do not treat a popular pattern as automatically appropriate.
```

5. Review AI-Generated Code

```
Review this patch as if you own the production incident that
could result from it:
[diff or file paths]

Check correctness, state ownership, concurrency, error handling,
security/privacy, performance, accessibility, and platform
boundaries.

Return only actionable findings, ranked by severity. For every
finding include:
- evidence in the code,
- user or system impact,
- a minimal correction,
- a test that would catch the regression.

If no finding is justified, say so and list the remaining
unverified assumptions.
```

6. Design a Test Matrix

```
Create a risk-based test matrix for [change].

Supported platforms: [macOS/iPhone/iPad/visionOS/etc.]
Critical workflows: [workflow list]
Known regression risk: [risk list]

Include manual tests, automated tests, test data, expected
result, and failure signal. Prioritise the smallest set that
proves the requested behaviour.
Separate build proof from runtime proof. Mark tests that require
a physical device, external account, or reviewer action.
```

7. Investigate a Production Issue

```
Help triage this production issue without exposing sensitive
data.

Symptoms: [user impact and scope]
Timeline: [first seen, release/build, frequency]
Signals: [redacted logs, metrics, crash signatures]
Recent changes: [deploys/configuration/dependencies]

Produce:
1. ranked hypotheses with evidence for and against each,
2. safe next diagnostic steps,
3. containment options that do not hide the root cause,
4. a communication draft for affected users,
5. exit criteria for declaring the incident resolved.
```

8. Security and Privacy Threat Review

```
Threat-model [feature or workflow].

Assets to protect: [data, credentials, files, money,
availability].
Actors and trust boundaries: [users, app, server, third parties].
Data flow: [brief flow].

Identify realistic threats, not generic checklists. For each,
provide likelihood, impact, existing controls, smallest
mitigation, and verification.
Call out data that must never be placed in an AI prompt or log.
```

9. Extract Domain Knowledge Before Building

```
I am building [feature] for [domain and users].

Help me identify the domain knowledge needed before
implementation. Ask the most important questions about workflow,
terminology, regulations, edge cases, incentives, and failure
consequences.

Then create a glossary, assumptions list, and a set of scenarios
that a domain expert should validate. Do not substitute generic
software conventions for domain facts.
```

10. Turn Work Into Portfolio Evidence

```
Turn this completed work into credible public engineering
evidence:
[project/change summary]

Audience: [hiring manager, technical peer, user community].
Constraints: do not disclose confidential information or
overstate outcomes.

Create:
- a concise project summary,
- the problem, decision, and trade-off,
- what I personally owned,
- verification evidence and measured outcome (only if supplied),
- lessons learned and next step.

Clearly label any claim that needs measurement or approval before
publication.
```

Quality Gate Before Acting on an Answer

- Is the goal, context, and definition of done explicit?
- Are facts separated from assumptions?
- Does the recommendation preserve stated constraints and existing behaviour?
- Is there a concrete verification step for every important claim?
- Did the answer avoid secrets, personal data, and unsupported certainty?
- Does a human remain accountable for architecture, security, release decisions, and user impact?